

Debugging (or) Optimization Task.

1. Deadlock Debugging:

Process P1:

lock(R1)
wait(100ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)

Process P2:

lock(R2)
wait(100ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)

Soln: Deadlock Condition:

- *. Process P1 locks R1 and waits for R2.
- *. Process P2 locks R2 and waits for R1.
- *. Both processes wait for each other indefinitely.
- *. This creates a Circular Wait condition, causing a deadlock.

Optimisation:

- *. Use a fixed resource allocation order.
- *. Make both processes lock R1 first, then R2.
- *. This removes the circular wait condition and prevents deadlock.

2. CPU Utilisation Problem:

A system shows low CPU utilization (30%) even though processes are ready. You analyze the scheduler log and find frequent transitions:

ready $\rightarrow$ waiting $\rightarrow$ ready $\rightarrow$ waiting $\rightarrow$ ready...

Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

Soln:

Cause:

- *. Processes frequently enter the waiting state ~~due~~ due to I/O operations.
- *. The CPU remains idle while waiting for I/O completion.
- *. This results in low CPU utilisation.

Optimisation:

- *. Use asynchronous I/O or improve I/O scheduling.
- *. Increase the degree of multiprogramming so another process executes while one waits for I/O.
- *. This improves CPU utilization and system performance.

3. Memory Bottleneck Debugging:

A multi-threaded program frequently encounters page faults. Identify the memory issue causing performance degradation and process two optimisations.

Soln:

Memory Issue:

- *. The application uses a 150 MB heap and 25 threads, each with a 1 MB stack.
- *. Frequent page faults occur because memory is accessed inefficiently.

Optimisations:

- *. Reduce the number of threads or reduce the stack size for each thread
- *. Improve memory allocation and locality of reference to reduce page faults.

4. Starvation Debugging:

Process P5 in Q3 never gets CPU time in a multilevel queue scheduler. Debug the root cause and suggest an optimisation.

Soln:

Root Cause:

- *. P5 is the lowest priority queue (Q3).

- *. Higher-priority queues (Q1 and Q2) continuously receive processes.
- *. Therefore, P5 never gets CPU time, leading to starvation.

Optimization :

- *. Apply Aging
- *. Gradually increase the priority of processes waiting for a long time.
- *. This ensures fairness and prevents starvation.

5. Context-Switch Overhead Debugging :

A server performs 20,000 context switches per second. Explain why context switching is high and propose two optimizations.

Soln. Reason :

- *. The time quantum is 1ms, while the average CPU burst time is 2ms.
- *. Frequent I/O Operations also cause many context switches.
- *. This increases CPU overhead and reduce performance.

Optimisations :

- *. Increase the time quantum (for example, from 1ms to 4ms).
- *. Reduce unnecessary context switches by improving I/O handling, such as using buffering or asynchronous I/O.

Result :

- *. Fewer context switches.
- *. Better CPU utilisation.
- *. Improved overall system performance.